

Trabalho 2

December 29, 2024

1 Classificação de Florestas

Trabalho Realizado por:

Tiago Pinto 54718 & João Loios 55469

1.1 Introdução

O nosso objetivo com este trabalho, é participar no desafio na plataforma kaggle (www.kaggle.com), que consiste em submeter modelos que classificam florestas, com base em informação cartográfica. O desafio está disponível em <https://www.kaggle.com/t/440cc105d3214fdf8e73961179081527>.

1.2 Implementação

1.2.1 Caracterização do conjunto de dados

Os dados são compostos por 2 subconjuntos: o conjunto de treino (train.csv) e o conjunto de teste (test.csv). É usado pandas para ser possível a transformação dos dados nos ficheiros “.csv” em DataFrames. Eles estão guardados da seguinte maneira:

```
[1]: # imports
import pandas as pd
from sklearn.ensemble import GradientBoostingClassifier, \
    RandomForestClassifier, ExtraTreesClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.feature_selection import SelectFromModel
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.model_selection import GridSearchCV
```

```
[2]: train_data = pd.read_csv('train.csv')
test_data = pd.read_csv('test.csv')
```

1.2.2 Análise do Conjunto de Dados

De modo a fazer o diagnostico final é necessário conhecer os dados do conjunto.

- id - identificador do exemplo
- elevação - elevação em metros
- aspeto- Aspeto em graus azimuth

- inclinação - inclinação em graus
- dh_agua - Distância horizontal até ao recurso hídrico superficial mais próximo
- dv_agua - Distância vertical até ao recurso hídrico superficial mais próximo
- dh_estrada - Distância horizontal até à estrada mais próxima
- sombra_9 - Índice de sombra às 9h
- sombra_12 - Índice de sombra às 12h
- sombra_15 - Índice de sombra às 15h
- dh_incendio - Distância horizontal até ao ponto de ignição de incêndios florestais mais próximo
- area - Tipo de área selvagem
- solo - Tipo de solo

1.2.3 Split dos Dados

Após a leitura dos ficheiros .csv separamos os atributos das classes e eliminamos a coluna id. Depois criamos os conjuntos de teste e o conjunto de ID usado no output.

```
[3]: X = train_data.drop(columns=['id', 'floresta'])
y = train_data['floresta']

test_ids = test_data['id']
X_test = test_data.drop(columns=['id'])
```

1.2.4 Experiências/Pesquisas para encontrar os melhores algoritmos

- Após o Split fomos testar diferentes algoritmos, com os seus parâmetros padrões, para termos um ponto de partida de qual algoritmo poderia ser o melhor a ser usado. Para essa seleção olhamos para a accuracy de cada um.
- Testamos diversos modelos, entre os quais, Decision Tree, Gradient Boosting, KNN, Random Forest, Extra Trees e Voting Classifier.
- Visto que o Voting Classifier é um modelo que combina previsões de vários algoritmos base para produzir uma previsão final mais robusta e precisa. Os algoritmos que usamos foram o Random Forest, o Extra Trees e o Gradient Boosting.
- Para obter os melhores resultados possíveis, criamos uma função que vê quais são os melhores parâmetros para cada um dos algoritmos. Esta função utiliza um grid search combinado com validação cruzada.

```
[4]: def melhorParametros(modelo, X, y):
    #ajustar os parametros
    parameters = {'n_estimators': [200, 500],
                  'max_depth': [50, 100, 150]}

    grid_search = GridSearchCV(modelo, parameters, cv=5)
    grid_search.fit(X, y)

    return grid_search.best_params_
```

1.2.5 Accuracy Obtidas

Modelo	Accuracy
Decision Tree	0.75
Gradient Boosting	0.81
KNN	0.75
Random Forest	0.86
Extra Trees	0.88
Voting Classifier	0.88

Conclusões Como podemos concluir, onde obtivemos melhores resultados foram no Extra Trees e no Voting Classifier. Sendo assim, procedemos a realizar o restante do trabalho usando estes dois modelos de base.

1.2.6 Construção dos Modelos

- Para desenvolver os modelos, inicialmente começamos por utilizar a função `SelectFromModel`, que possui o parâmetro `threshold` que permite eliminar os atributos menos significantes nos modelos.
- No caso do Voting Classifier, apenas usamos o `SelectFromModel`, para o Extra Trees, visto que ao utilizarmos os atributos padrões, este era o que estava a obter melhores resultados.
- Depois, construímos o restante do código normalmente, utilizando os melhores parâmetros para cada um dos algoritmos.
- Os modelos foram construídos numa função que retorna o `predict`.

Voting Classifier

```
[5]: # usar dados de treino
def usarTreinoVotingClf(X, y):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
    ↪random_state=5)

    extraTree = ExtraTreesClassifier(n_estimators=500, max_depth=100)
    extraTree.fit(X_train, y_train)

    sfm = SelectFromModel(extraTree, threshold=0.05)
    sfm.fit(X_train, y_train)
    X_train_transformados = sfm.transform(X_train)
    X_test_transformados = sfm.transform(X_test)

    randomForest = RandomForestClassifier(n_estimators=500, max_depth=100)
    gradientBoosting = GradientBoostingClassifier(n_estimators=300,
    ↪max_depth=100)

    #Inicialização do voting classifier
    VotingClass = VotingClassifier(estimators=[('rf1', randomForest), ('rf2',
    ↪gradientBoosting), ('rf3', extraTree)], voting='hard', weights=[3,1,5])
    VotingClass.fit(X_train_transformados, y_train)
    predictions = VotingClass.predict(X_test_transformados)
```

```
return y_test, predictions
```

```
[6]: # usar dados de teste
def usarTesteVotingClf(X, X_test, y):

    extraTree = ExtraTreesClassifier(n_estimators=500, max_depth=100)
    extraTree.fit(X, y)

    sfm = SelectFromModel(extraTree, threshold=0.05)
    sfm.fit(X, y)
    X_train_transformados = sfm.transform(X)
    X_test_transformados = sfm.transform(X_test)

    randomForest = RandomForestClassifier(n_estimators=500, max_depth=100)
    gradientBoosting = GradientBoostingClassifier(n_estimators=300,
↪max_depth=100)

    #Inicialização do voting classifier
    VotingClass = VotingClassifier(estimators=[('rf1', randomForest), ('rf2',
↪gradientBoosting), ('rf3', extraTree)], voting='hard', weights=[3,1,5])
    VotingClass.fit(X_train_transformados, y)
    predictions = VotingClass.predict(X_test_transformados)

    return predictions
```

Extra Trees

```
[7]: # usar dados de treino
def usarTreinoExtraTrees(X, y):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
↪random_state=5)

    extraTree = ExtraTreesClassifier(n_estimators=500, max_depth=100)
    extraTree.fit(X_train, y_train)

    sfm = SelectFromModel(extraTree, threshold=0.05)
    sfm.fit(X_train, y_train)
    X_train_transformados = sfm.transform(X_train)
    X_test_transformados = sfm.transform(X_test)

    #Inicialização do voting classifier
    extraTree.fit(X_train_transformados, y_train)
    predictions = extraTree.predict(X_test_transformados)

    return y_test, predictions
```

```
[8]: # usar dados de teste
def usarTesteExtraTrees(X, X_test, y):

    extraTree = ExtraTreesClassifier(n_estimators=500, max_depth=100)
    extraTree.fit(X, y)

    sfm = SelectFromModel(extraTree, threshold=0.05)
    sfm.fit(X, y)
    X_train_transformados = sfm.transform(X)
    X_test_transformados = sfm.transform(X_test)

    #Inicialização do voting classifier
    extraTree.fit(X_train_transformados, y)
    predictions = extraTree.predict(X_test_transformados)

    return predictions
```

1.2.7 Produção dos Ficheiros de Submissão

Voting Classifier

```
[ ]: # Para usar o voting classifier com o conjunto de treino
y_true,y_pred = usarTreinoVotingClf(X, y)
print("Precisão com o conjunto de treino:", accuracy_score(y_true,y_pred))

# com o conjunto de teste
y_pred = usarTesteVotingClf(X, X_test, y)

output = pd.DataFrame({'id': test_ids,'floresta': y_pred})
output.to_csv('respostaVoting.csv', index=False)

print(f"Previsões salvas em: {'respostaVoting.csv'}")
```

Extra Trees

```
[ ]: #com o conjunto de treino
y_true,y_pred = usarTreinoExtraTrees(X, y)
print("Precisão com o conjunto de treino:", accuracy_score(y_true,y_pred))

#com o conjunto de teste
y_pred = usarTesteExtraTrees(X, X_test, y)

output = pd.DataFrame({'id': test_ids,'floresta': y_pred})
output.to_csv('respostaExtraTree.csv', index=False)

print(f"Previsões salvas em: {'respostaExtraTree.csv'}")
```

1.2.8 Desempenho das nossas 4 melhores submissões

Submissão	Public Score	Private Score	Modelo
1º	0.87899	0.87034	Voting Classifier
2º	0.87865	0.87034	Extra Trees
3º	0.87695	0.86777	Voting Classifier
4º	0.87627	0.87163	Voting Classifier

Conclusão Como podemos ver, segundo a tabela, o modelo que obteve os melhores desempenhos foi o Voting Classifier, apesar do Extra Trees, também ter conseguido valores muito próximos.

1.3 Conclusão

- Por fim, acreditamos que o trabalho foi bem conseguido apesar de não termos conseguido melhores resultados. Tivemos diversos momentos de paragem, ao longo do projeto, pois não sabíamos como aumentar mais a exatidão das nossas submissões, mas graças a alguma pesquisa conseguimos obter melhores resultados.
- Por fim, apesar de desafiador, gostamos de realizar este trabalho, e pretendemos melhorar os nossos conhecimentos e adquirir mais, futuramente.